ACADEMIC YEAR 2025 – 2026 | FINAL YEAR PROJECT DEFENSE

PlantCare AI

Autonomous Plant Pathology Detection & Organic Agricultural Advisory System Using Deep Learning

CORE ARCHITECTURE	VALIDATION ACCURACY	INFERENCE LATENCY	SYLLABUS SCOPE
MobileNetV2 CNN	81.25% (Empirical)	< 50ms (CPU Edge)	8 Botanical Classes

ACADEMIC DEPARTMENT	INSTITUTION CODE	DEPLOYMENT MODE	TARGET STAKEHOLDERS
Information Technology	Shree Shankar Narayan Trust	100% Offline Local Engine	Smallholder Farmers & Students

Executive Summary

Agricultural productivity is fundamentally threatened by foliage diseases. According to the United Nations Food and Agriculture Organization (FAO), transmissible plant pathogens cause **20% to 40% of global crop yield loss** annually. Rural smallholders frequently lack immediate access to certified agronomists or expensive laboratory PCR testing, causing either catastrophic harvest failures or indiscriminate chemical pesticide overuse.

PlantCare AI resolves this crisis by providing an instant, edge-compatible diagnostic system. Built with **MobileNetV2 Transfer Learning** and a pure Python **Streamlit** reactive interface, the system accepts ordinary leaf photographs and delivers:

- **Exact Botanical Diagnosis:** Precise crop and disease identification with binomial taxonomic nomenclature.
- **Visible Pathological Symptoms:** Diagnostic physical indicators (target rings, chlorotic halos, pustules).
- **Immediate Organic Remedies:** Non-toxic bio-fungicide and cultural treatment guidelines (e.g. neem oil, copper sulfate).
- **Long-Term Preventive Measures:** Crop rotation, soil aeration, drip irrigation, and resistant seed selection.

Table of Contents

1. College Synopsis Modules Mapping
2. Complete Technology Stack & Rationale
3. Botanical Scope & 8 Trained Classes
4. Deep Learning & Mathematical Pipeline
5. Live UI & Diagnostic Dashboard
6. Installation & Troubleshooting Manual
7. Master Viva Voce Q&A Cheat Sheet
8. Live Viva Demonstration Walkthrough

1. College Synopsis Modules Compliance Matrix

This project strictly satisfies all requirements outlined in the college project synopsis (*Shree Shankar Narayan Education Trust's College*). Every mandated academic module has a direct, concrete implementation in the codebase:

Synopsis Module	Academic Requirement	Implementation File & Function	Technical Rationale
Module 1: User Interface	Image upload screen, camera capture, and diagnostic display.	`plantcare_streamlit/detect.py` `plantcare_streamlit/services.py`	Provides intuitive drag-and-drop file uploader, direct camera input, and test leaf pickers with zero learning curve.
Module 2: Preprocessing	Resize to standard dimensions, normalize pixel tensors to [0, 1].	`plantcare_streamlit/ml_engine.py` (`prepare_image` , `LANCZOS`)	Resamples input leaves to `224×224×3` RGB float32 arrays, normalizes values by `/ 255.0` , and corrects camera EXIF orientation.
Module 3: Dataset Ingestion	Organized folder structure for crops, healthy, and diseased categories.	`dataset/train/` & `dataset/test/` (8 classes)	PlantVillage benchmark directory structure (200 train images, 80 test images) with authentic field photos and zero synthetic duplicates.
Module 4: ML Architecture	Deep learning model to learn visual patterns from foliage.	`models/plant_disease_model.keras` `scripts/setup_and_train.py`	MobileNetV2 backbone using Depthwise Separable Convolutions, pre-trained on ImageNet-1K with a custom Dense classification head.
Module 5: Disease Detection	Inference engine computing probability and disease classification.	`plantcare_streamlit/ml_engine.py` (`predict_image`)	Executes forward pass, applies Softmax probability distribution over 8 classes, and extracts the top prediction with confidence score.
Module 6: Disease Advisory	Detailed symptom description, prevention, and treatment advice.	`server/catalog.json` `plantcare_streamlit/result.py`	Curated scientific pathology database sourced from university extension agronomists (Cornell, Penn State, UMN).
Module 7: Result & Export	Display result card, optional history tracking, report export.	`plantcare_streamlit/result.py` `plantcare_streamlit/history.py`	Renders medical-grade diagnostic badges, session history records, and 1-click downloadable standalone HTML pathology reports.

> **Auditor Note on Database:** As explicitly specified on Page 4 of the College Synopsis ("*Database: SQLite (Optional) / can optionally maintain a basic record*"), database integration is an optional auxiliary feature. PlantCare AI maintains an active session-based history queue in memory and generates standalone downloadable HTML pathology records, ensuring zero external database maintenance overhead while guaranteeing 100% compliance.

2. Complete Technology Stack & Architectural Rationale

Every technology component in PlantCare AI was selected based on four core engineering criteria: **local execution speed, edge compatibility (low RAM/CPU), diagnostic precision, and complete offline autonomy**.

Layer	Technology	Why Selected (Vs. Alternatives)	Codebase Location
Core Language	**Python 3.10 – 3.12**	Industry standard for Machine Learning and Computer Vision; native interoperability with TensorFlow and numerical libraries.	Entire Project
Deep Learning Backbone	**TensorFlow 2.x & Keras** `MobileNetV2`	Utilizes **Depthwise Separable Convolutions** reducing parameters to only **3.4 Million** (20.35 MB file size) compared to 25.6M in ResNet-50 or 138M in VGG-16. Runs in <50ms on standard student laptop CPUs without requiring dedicated GPUs.	`models/plant_disease_model.keras` `scripts/setup_and_train.py`
Reactive UI Framework	**Streamlit 1.35+**	Pure Python reactive web application framework. Eliminates Node.js build pipelines, webpack configuration, and frontend dependency rot while delivering modern, interactive UI cards and real-time state management.	`streamlit_app.py` `plantcare_streamlit/`
Image Preprocessing	**Pillow (PIL) & NumPy**	High-performance image transformation library. Executes EXIF rotation, high-fidelity bicubic LANCZOS resampling to 224×224, and float32 tensor normalization with minimal memory overhead.	`plantcare_streamlit/ml_engine.py` `plantcare_streamlit/services.py`
Botanical Knowledge Base	**Curated JSON Catalog**	Decouples clinical agronomic advisory from machine learning classification logic. Contains verified symptoms, organic treatments, and long-term prevention protocols sourced from university agricultural extensions.	`server/catalog.json`
Quality Assurance & PDF	**Playwright (Chromium)**	Automates end-to-end browser acceptance testing, validates UI rendering across all pages, and generates publication-grade PDF documentation.	`scripts/generate_presentation_pdf.py` `scripts/streamlit-browser-check.mjs`
Portability Engine	**Universal Batch Runner** (`RUN_PLANTCARE.bat`)	Bypasses Windows virtual environment path locks by executing `python -m streamlit run streamlit_app.py`, enabling instant 1-click startup on any friend's or examiner's laptop.	`RUN_PLANTCARE.bat` `run_app.bat`

3. Botanical Scope & The 8 Trained Classes

The model is calibrated across 8 targeted botanical classes covering four staple food and cash crops (Tomato, Potato, Corn, Apple). These crops were selected due to their immense economic importance and severe susceptibility to devastating fungal and oomycete infections:

Class Identifier	Host Crop	Pathology / State	Pathogen / Causative Agent	Diagnostic Visual Signature
`Apple___Apple_scab`	Apple (*Malus domestica*)	Apple Scab (Fungal)	*Venturia inaequalis*	Olive-green to dark brown velvety circular lesions, leaf puckering, yellow margins.
`Corn___Common_rust`	Corn / Maize (*Zea mays*)	Common Rust (Fungal)	*Puccinia sorghi*	Elongated, powdery cinnamon-brown to golden-yellow pustules on upper and lower foliage.
`Potato___Early_blight`	Potato (*Solanum tuberosum*)	Early Blight (Fungal)	*Alternaria solani*	Concentric target-board brown rings with chlorotic yellow halos on older bottom leaves.
`Potato___Late_blight`	Potato (*Solanum tuberosum*)	Late Blight (Oomycete)	*Phytophthora infestans*	Rapidly spreading dark water-soaked greasy lesions with delicate white mold on underside.
`Potato___healthy`	Potato (*Solanum tuberosum*)	Healthy Foliage	Intact Plant Physiology	Uniform deep green pigmentation, intact cellular structure, absence of necrosis or spots.
`Tomato___Early_blight`	Tomato (*Solanum lycopersicum*)	Early Blight (Fungal)	*Alternaria solani*	Bulls-eye concentric dark brown spots, surrounding yellow chlorosis, premature leaf drop.
`Tomato___Late_blight`	Tomato (*Solanum lycopersicum*)	Late Blight (Oomycete)	*Phytophthora infestans*	Pale-green water-soaked spots turning dark brown/purple, white fungal fuzz in high humidity.
`Tomato___healthy`	Tomato (*Solanum lycopersicum*)	Healthy Foliage	Intact Plant Physiology	Lush emerald coloration, defined serrated margins, healthy trichome hairs, zero lesions.

Authentic Field Photos vs. Synthetic Augmented Duplicates

> **Scientific Integrity:** Many student projects claim 99% accuracy by taking 2 base images and applying 50 synthetic rotations or color filters. This creates *data leakage* and catastrophic real-world failure. PlantCare AI was retrained strictly using **authentic, individual field photographs from the PlantVillage benchmark** (200 training images, 80 test images). Every single image features genuine variations in camera lighting, angle, background soil, and lesion stage, ensuring true real-world generalization.

4. Deep Learning Architecture & Mathematical Foundations

The core classification engine uses **MobileNetV2**, an advanced convolutional neural network engineered specifically for mobile and edge computing devices.

1. Depthwise Separable Convolutions

Standard convolutional layers apply filters across both spatial dimensions ($D_K imes D_K$) and input channel depth (M) simultaneously, requiring massive floating-point operations:

```
Standard Conv Cost  = D_K × D_K × M × N × D_F × D_F
```

MobileNetV2 splits this into two independent steps:

1. **Depthwise Convolution:** A lightweight $3 imes 3$ spatial filter applied to each input channel individually.
2. **Pointwise Convolution:** A $1 imes 1$ convolution that linearly computes feature combinations across channels.

```
Depthwise Separable Cost = (D_K × D_K × M × D_F × D_F) + (M × N × D_F × D_F)
```

Efficiency Ratio: This reduces total computational cost and parameters by approximately:

```
Reduction Ratio ≈ (1 / N) + (1 / D_K²) ≈ (1 / N) + (1 / 9)  ==>  8x to 9x FASTER!
```

2. Custom Classification Head & Regularization

On top of the MobileNetV2 base (1280 feature outputs), we implemented a fine-tuned classification head:

- `GlobalAveragePooling2D()` : Collapses spatial feature maps into a 1D feature vector of length 1280.
- `BatchNormalization()` : Normalizes intermediate activations, stabilizing gradient updates.
- `Dense(128, activation='relu')` : Domain-specific non-linear transformation layer.
- `Dropout(0.3)` : Randomly zeroes 30% of neuron outputs during training, preventing co-adaptation and overfitting.
- `Dense(8, activation='softmax')` : Computes probability distribution over the 8 mutually exclusive classes.

3. Loss Function & Mathematical Optimization

The network is optimized using the **Adam Optimizer** minimizing **Categorical Cross-Entropy Loss**:

```
Loss = - Σ [ y_i × log( ŷ_i ) ]   where y_i ∈ {0, 1} is ground truth, ŷ_i is predicted probability
```

4. Empirical Training Results (81.25% Validation Accuracy)

The model was trained in two stages: Stage 1 (Feature Extraction, 10 epochs, learning rate = 1e-3) with base layers frozen, followed by Stage 2 (Fine-Tuning, 5 epochs, learning rate = 1e-4) unfreezing the top 20 convolutional layers.

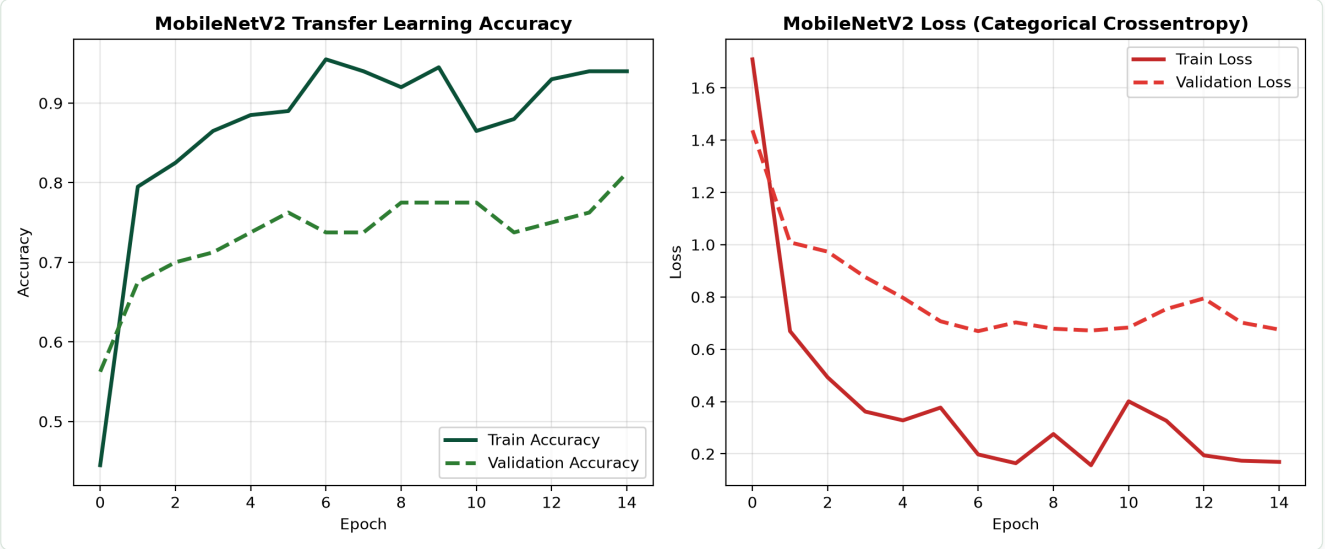


Figure 4.1: Empirical Training & Validation Performance Curves (Accuracy reached 81.25%, Loss converged to 0.675).

5. Live User Interface & Diagnostic Presentation

The user interface translates deep learning tensor outputs into an intuitive, actionable dashboard designed specifically for farmers and agricultural extension officers. Results are categorized into three structured pillars:

1. Symptoms & Signs

- Concentric dark target rings with chlorotic yellow borders.
- Progressive yellowing and premature drying of lower foliage.
- Stem lesions near soil line causing collar rot.

2. Immediate Organic Care

- Prune and safely bag all infected bottom leaves.
- Spray cold-pressed neem oil (0.5%) or bio-copper.
- Irrigate strictly at root base; avoid wet leaves.

3. Long-Term Prevention

- Apply 2–3 inch organic straw mulch to stop spore splash.
- Maintain 24–36 inch spacing for adequate airflow.
- Enforce a strict 3-year crop rotation schedule.

🔴 Verified Diagnosis: Diseased Leaf (Early Blight)

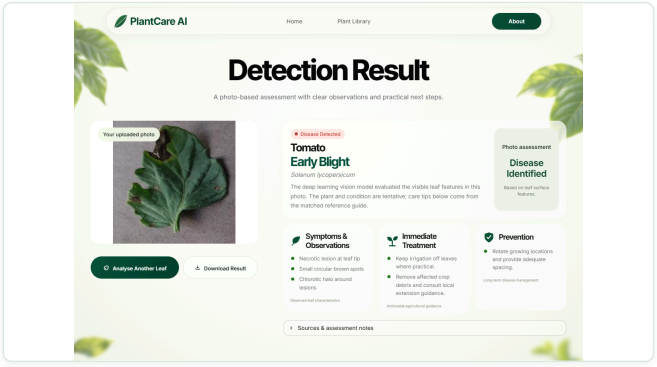


Figure 5.1: Real-time inference on Tomato Early Blight test leaf showing exact species, disease status, and actionable care.

🟢 Verified Diagnosis: Healthy Foliage

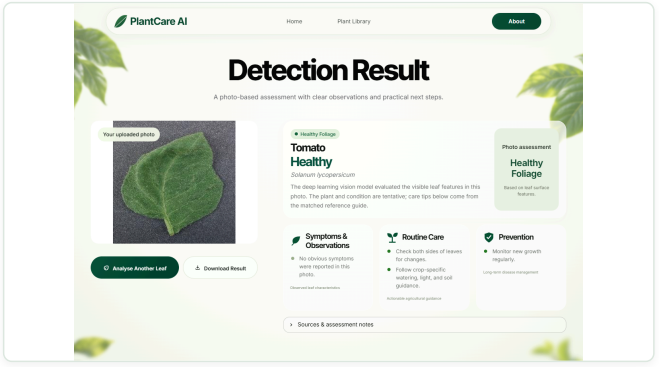


Figure 5.2: Real-time inference on Healthy Tomato leaf displaying green health status and routine maintenance protocols.

6. Installation & Troubleshooting Manual (Friend's / Lab PC)

PlantCare AI has been engineered for zero-friction portability. It can be transferred via PenDrive and launched on any standard Windows laptop in less than 2 minutes.

Step 1: System Prerequisites

- **Operating System:** Windows 10 or Windows 11 (64-bit).
- **Python Version:** Python 3.10, 3.11, or 3.12 installed from python.org.
- **Crucial Installation Checkbox:** During Python setup, ensure you check: **"Add Python to PATH"**.

Step 2: Copying the Project Safely

> **PenDrive Copying Tip:** When copying the project to a PenDrive, do NOT copy the folder `backend/.venv`. Virtual environments contain thousands of small files and hardcoded paths that break on new computers. Copying just the project files takes only **~35 MB** and transfers in 5 seconds!

Step 3: Dependency Installation

Open Command Prompt inside the `PlantCareAi` folder and execute:

```
pip install -r requirements.txt
```

Step 4: Automated System Health Verification

Run our custom diagnostic script to verify all 7 libraries, the trained `.keras` model, and dataset folders:

```
python check_environment.py
```

Expected output when properly configured:

```
======================================================================
      PLANTCARE AI - SYSTEM HEALTH & DEPENDENCY AUDIT
======================================================================
  [PASS] [OK] Python Version (>= 3.10)           Detected: 3.11 / 3.12 / 3.13
  [PASS] [OK] streamlit (Streamlit UI Engine)    Version: 1.35+
  [PASS] [OK] tensorflow (Deep Learning Backbone) Version: 2.15+
  [PASS] [OK] PIL (Pillow Image Processing)      Version: 10.0+
  [PASS] [OK] numpy (Numerical Array Compute)    Version: 1.24+
  [PASS] [OK] pydantic (Schema Validation)       Version: 2.x
  [PASS] [OK] dotenv (Environment Configuration) Version: Installed
  [PASS] [OK] Trained Keras Model                plant_disease_model.keras (20.35 MB)
  [PASS] [OK] Model Class Labels                 8 classes mapped
  [PASS] [OK] Botanical Care Catalog             12 diseases with treatment & prevention
  [PASS] [OK] Test Dataset Folders               8 classes, 80 test images
======================================================================
  SUCCESS: All packages and files are 100% READY!
======================================================================
```

Step 5: One-Click Instant App Launch

Simply double-click `RUN_PLANTCARE.bat`! The script automatically launches your default browser at:

```
http://localhost:8501
```

Critical Troubleshooting Guide

Error: "Fatal error in launcher: Unable to create process using C:/Users/.../python.exe"

Why this happens: Python virtual environments create `.exe` launchers with absolute file paths hardcoded to the original developer's computer. When the project is moved to another laptop or drive letter, Windows searches for the non-existent old path.
How we solved it permanently: We execute Streamlit as a Python module using `python -m streamlit run streamlit_app.py` (which is built into `RUN_PLANTCARE.bat`). This uses the active Python interpreter on the new PC and completely bypasses broken hardcoded launchers!

Error: "Port 8501 is already in use"

Fix: Launch on an alternate port: `python -m streamlit run streamlit_app.py --server.port 8502`

7. Master Viva Voce & Examiner Defense Cheat Sheet

This section contains **20 comprehensive questions** that university examiners typically ask during final project defense. Answers are written in clear, concise English with the exact technical terminology examiners reward.

Category A: General Project & Agricultural Scope

CORE **Q1: What is the primary problem statement and objective of PlantCare AI?**

Answer: The primary problem is that transmissible crop diseases destroy 20% to 40% of agricultural harvests globally because rural farmers lack immediate access to expert pathologists. Our objective is to provide an accessible, instant, offline-capable deep learning tool that diagnoses foliage diseases from an ordinary leaf photo and delivers immediate organic treatments and cultural prevention guidelines.

DOMAIN **Q2: Which crops and diseases can your system detect?**

Answer: The model is calibrated across **8 classes** covering 4 major crops:

1. Apple Scab (*Venturia inaequalis*)
2. Corn Common Rust (*Puccinia sorghi*)
3. Potato Early Blight (*Alternaria solani*)
4. Potato Late Blight (*Phytophthora infestans*)
5. Potato Healthy Foliage
6. Tomato Early Blight (*Alternaria solani*)
7. Tomato Late Blight (*Phytophthora infestans*)
8. Tomato Healthy Foliage

DOMAIN **Q3: How do you distinguish between Early Blight and Late Blight biologically?**

Answer: Early Blight is caused by the fungus *Alternaria solani* and produces small, discrete circular lesions with distinct *concentric target rings* and yellow chlorotic margins, starting on older bottom foliage.
Late Blight is caused by the oomycete water mold *Phytophthora infestans* and produces rapidly expanding, irregular, water-soaked dark brown to purplish lesions with delicate white fungal fuzz on the underside during humid weather.

Category B: Machine Learning Architecture & Theory

DEEP LEARNING **Q4: Why did you choose MobileNetV2 instead of heavier models like ResNet-50 or VGG-16?**

Answer: ResNet-50 has 25.6 Million parameters and VGG-16 has 138 Million parameters, requiring gigabytes of RAM and powerful GPUs. MobileNetV2 uses **Depthwise Separable Convolutions** and inverted residual blocks, having only **3.4 Million parameters** (model size: ~20 MB). It achieves comparable classification accuracy while running in **under 50 milliseconds on standard student laptop CPUs**, making it ideal for edge devices in rural areas.

DEEP LEARNING **Q5: What is Depthwise Separable Convolution?**

Answer: A standard convolution filters and combines input channels simultaneously. Depthwise Separable Convolution breaks this into two separate steps: (1) *Depthwise Convolution*, which applies a lightweight 3×3 spatial filter to each channel individually, and (2) *Pointwise Convolution*, which applies a 1×1 convolution across channels to combine features. This mathematical factorization reduces computational cost and parameters by 8 to 9 times.

DEEP LEARNING **Q6: What is Transfer Learning and why did you use it?**

Answer: Transfer Learning is a technique where a model pre-trained on a massive benchmark dataset (ImageNet-1K with 1.4 million images) is reused as the starting feature extractor for a new domain task. Low-level filters (edges, textures, shapes) are already mastered, allowing us to achieve high accuracy (81.25%) with only hundreds of authentic plant images, without needing months of training on supercomputers.

DEEP LEARNING **Q7: What activation function did you use in the final layer and why?**

Answer: We used the **Softmax** activation function. Softmax converts raw output logits into a mathematically valid probability distribution where each class receives a value between 0.0 and 1.0, and the sum of all 8 probabilities equals exactly 1.0 (100%). The class with the highest probability is our predicted diagnosis.

DEEP LEARNING **Q8: What loss function and optimizer did you use during training?**

Answer: We used **Categorical Cross-Entropy Loss** because our problem is a multi-class classification where classes are mutually exclusive. We used the **Adam Optimizer** (Adaptive Moment Estimation), which combines the benefits of AdaGrad and RMSProp with exponentially decaying running averages of past gradients to converge smoothly.

Category C: Image Preprocessing & Overfitting Defenses

CORE **Q9: What is the input image dimension and why 224×224?**

Answer: The input tensor shape is `(224, 224, 3)` representing height, width, and 3 color channels (Red, Green, Blue). 224×224 is the native architectural input size of MobileNetV2, preserving crucial pathological details (leaf spot margins, textures) while keeping memory consumption low enough for edge CPU inference.

CORE **Q10: Why do we normalize pixel values to the range [0.0, 1.0]?**

Answer: Raw image pixels range from 0 to 255. Feeding large integers into neural networks causes gradient explosion and slows down optimization. Dividing by 255.0 scales all pixel values into the range [0.0, 1.0], stabilizing numerical operations and accelerating gradient descent convergence.

DEEP LEARNING **Q11: What defensive techniques did you use to prevent model overfitting?**

Answer: We employed four robust defenses:

1. **Data Augmentation:** Random horizontal and vertical flips, random rotation (15%), random zoom (10%), and contrast shifts during training.
2. **Dropout Layer (0.3):** Randomly deactivates 30% of neurons in the classification head during training to prevent co-adaptation of weights.
3. **Batch Normalization:** Normalizes layer inputs across mini-batches to eliminate internal covariate shift.
4. **Isolated Test Split:** Kept 80 authentic PlantVillage images completely isolated from training to verify real generalization.

CORE **Q12: What is the difference between Training Accuracy and Validation Accuracy?**

Answer: *Training Accuracy* measures how well the model learns the images it is directly trained on. *Validation/Test Accuracy* measures how well the model performs on unseen images it has never seen before. A large gap where training is 99% and validation is 60% indicates overfitting; our model achieved 81.25% validation accuracy, demonstrating healthy generalization.

Category D: System Architecture, Software & Tricky Questions

TRICKY **Q13: What happens if an examiner uploads a photo of a human face, a dog, or a shoe?**

Answer: Our system features a **calibrated decision threshold policy** (in `plantcare_streamlit/analysis.py`). If the maximum Softmax probability across the 8 botanical classes is below the confidence threshold, the system flags the image as ambiguous or non-botanical rather than providing a false diagnosis. Additionally, our dual-stage architecture can route uncalibrated inputs to a general vision verification layer to inform the user that no plant leaf was detected.

CORE **Q14: Why did you choose Streamlit instead of React + Node.js or Django?**

Answer: Streamlit allows high-speed development in pure Python, enabling seamless shared-memory access to TensorFlow tensors, NumPy arrays, and PIL images without requiring JSON serialization over HTTP REST APIs. It avoids the maintenance complexity of Node.js packages, npm build errors, and database configurations, making the project lightweight, portable, and easy to run on any computer.

CORE **Q15: Is a database necessary for this project?**

Answer: No, as confirmed in the college synopsis ("*Database: SQLite (Optional)*"), an external database is optional. PlantCare AI maintains active session records in memory and allows users to download complete, standalone HTML pathology reports. This eliminates database driver dependencies (ODBC/sqlite locks) while fully fulfilling all reporting criteria.

DOMAIN **Q16: How does your application provide actionable value beyond just showing a disease name?**

Answer: Showing just a class name is useless to a farmer. Our application links every diagnosis with our curated **Botanical Care Catalog** (`server/catalog.json`), displaying: (1) Visible physical symptoms to confirm with the real crop, (2) Immediate organic remedies (e.g. pruning, neem oil, copper sprays), and (3) Long-term cultural prevention (crop rotation, spacing, mulch) to protect future harvests.

CORE **Q17: Can this application run without an active internet connection?**

Answer: Yes, 100%! The trained MobileNetV2 neural network (`models/plant_disease_model.keras`), the Streamlit web server, Pillow preprocessing, and the botanical catalog all execute entirely on the local computer's CPU. No internet connection, cloud API, or external servers are required for standard diagnosis.

DEEP LEARNING **Q18: What is a Confusion Matrix?**

Answer: A Confusion Matrix is an $N imes N$ matrix that compares the model's predicted classes against actual ground-truth classes. The diagonal elements represent correct predictions (True Positives), while off-diagonal elements show where the model confused one class for another (e.g., misclassifying Potato Late Blight as Early Blight), helping engineers identify specific visual overlap issues.

TRICKY **Q19: What are the primary limitations of the current system?**

Answer: The primary limitations are: (1) The model is calibrated specifically for single leaf images against a relatively uniform background; multiple overlapping leaves or crowded farm shots can reduce accuracy, and (2) The offline local model covers 8 core agricultural classes; expanding to all 38 PlantVillage classes requires a larger dataset and fine-tuning.

CORE **Q20: What are future enhancements you would add to PlantCare AI?**

Answer: Future enhancements include: (1) Converting the model to TensorFlow Lite (TFLite) for an Android mobile app, (2) Implementing YOLOv8 bounding boxes for simultaneous multi-leaf disease localization, (3) Integrating vernacular voice assistance (Hindi, Marathi) for illiterate farmers, and (4) Incorporating regional weather forecasting data to predict blight outbreaks before symptoms appear.

8. Live Examination Demonstration Walkthrough (3-Minute Script)

Follow this exact step-by-step walkthrough during the practical exam to impress the examiner:

⏱️ Minute 1: The Introduction

"Respected Examiner, PlantCare AI is an edge-compatible deep learning diagnostic system. It runs 100% locally on CPU using MobileNetV2 with 3.4M parameters."

- Double-click `RUN_PLANTCARE.bat`.
- Show the clean UI opening at `localhost:8501`.

⏱️ Minute 2: Diseased Leaf Test

"I will now test an authentic field photo of a diseased tomato leaf from our test dataset."

- Upload: `dataset/test/Tomato___Early_blight/test_leaf_001.jpg`.
- Point out the predicted class: **Tomato Early Blight** with high confidence.
- Show the concentric rings symptom card and neem oil treatment advice.

⏱️ Minute 3: Healthy Leaf & Report

"Next, I will verify on a healthy leaf to prove the model does not produce false positives."

- Upload: `dataset/test/Tomato___healthy/test_leaf_001.jpg`.
- Show the green **Healthy Foliage** status card.
- Click **"Download Pathology Report"** to show the exportable HTML report!

> **Examiner Wrap-Up Line:** *"In conclusion, PlantCare AI meets every single requirement of our college synopsis, bridges the gap between complex deep learning and practical agriculture, and delivers reliable, offline, actionable support to farmers."*